

Incremental Copy Approach

Always Incremental Copy approach is recommended to copy the daily data.

- ➔ Imagine if we are getting billions of data daily and if we use the general copy approach, we will copy repeated data daily, to resolve this we will use incremental copy approach to copy only new data instead of all data.

Example, yesterday we had copied 1000's of customer data and today if we got new data into database and in total, we had old data and new data in the database, so we cannot copy old and new data daily. Instead of copying all the data we will copy only today's data.

- ➔ To use this approach, we will create a table like metadata table which might contain the schema like below

```
create Table IncrementalTable (  
    Inc_ID Int,  
    IncTable Varchar(100),  
    IncSchema Varchar(100),  
    IncFolderPath Varchar(100),  
    IncColumn Varchar(100),  
    IncValue Int  
)
```

- ➔ The above table is replica of the metadata table. So, we will insert the table's data into the IncrementalTable
- ➔ Here we are doing incremental copy approach by using the column ID as Incremental value.

Steps to create a Pipeline in synapse

Step 1:

Create a Lookup Activity to search for the tables and to copy data from the tables

- ➔ Under the Lookup Activity settings **Add two parameters**

Connection	Schema	Parameters		
New Delete				
☐	Name	Type	Default value	
☐	TableName	String	Value	
☐	SchemaName	String	Value	

- ➔ Now under the connection enter the table schema and table name dynamically by selecting the dataset.

➔ Next, configure the Dataset properties by entering the values of the metadata data or incremental table which is created in the database before. **i.e., IncrementalTable**

General Settings User properties

Source dataset * INC_SQL_DB_Setup [Open](#) [New](#) [Preview data](#) [Learn more](#)

Dataset properties

Name	Value	Type
TableName	IncrementalTable	string
SchemaName	dbo	string

First row only ☐

Step 2:

Create a ForEach Activity to copy the data from the lookup output tables by using iterative

Connect lookup to ForEach

ForEach

ForEach1

Activities
No activities

Batch count ①

Items

@activity('LookUp').output.value

Activity outputs

LookUp
LookUp activity output

LookUp count
Count of the rows

LookUp value array
Array of row data

Enter the value into item dynamically by selecting the option as per image

ForEach Activity configuration is done as per the above image, and item value is connected to the lookup output array.

In the output array will be in JSON format where there will be values array containing the tables inside the IncrementalTable

Step 3:

Now Add the Copy Activity Inside the ForEach Activity to copy the data into ADLSGen2

- ➔ Use the same Source connection used for the Lookup activity
- ➔ Now, instead of adding dataset properties manually add them dynamically from the lookup output as per below image

General Source Sink Mapping Settings User properties

Source dataset * INC_SQL_DB_Setup Open + New Preview data Learn more

Dataset properties

Name	Value
TableName	Value Add dynamic content [Alt+Shift+D]
SchemaName	Value

Clear contents

ForEach iterator Acti

Search

ForEach1 Current item

- For TableName: write table name followed by item() expression
- For SchemaName: Write schema name followed by item() expression

Pipeline expression builder

Add dynamic content below using any combination of the following expressions

@item().IncValue

IncValue is Column created in IncrementalTable

Pipeline expression builder

Add dynamic content below using any combination of the following expressions

@item().IncSchema

IncSchema is Column created in IncrementalTable

➔ After adding the values dynamically, source settings look like below

General Source Sink Mapping Settings User properties

Source dataset * INC_SQL_DB_Setup Open + New Preview data Learn more

Dataset properties

Name	Value	Type
TableName	@item().IncValue	string
SchemaName	@item().IncSchema	string

➔ Now configure the sink dataset by selecting ADLSGEN2 as integration database and format as CSV.

- Under the sink dataset settings add the parameters named FolderPath and again in connection select the file path dynamically as before

General Source **Sink** Mapping Settings User properties

Sink dataset * INC_COPY Open New [Learn more](#)

Dataset properties ⓘ

Name	Value
FolderPath	@item().IncFolderPath

- ➔ Once folder path is set, then the copied files will be stored in the respective path and configured ADLSGEN2, when the pipeline is run.
- ➔ Suppose if there are multiple tables added into the meta data table then for each loop will copy all the tables data into ADLSGEN2.

Step 4:

Now after copying the data, incremental copy comes into picture to use the same pipeline daily to copy new data daily to ADLSGEN2 in incremental fashion.

For this we can use two ways either by incremental ID based or date based. This document will help to copy the data by ID based.

Explanation to Logic:

When the data is copied into databases there will be maximum value of the data(rows) let's say for example, today we copied 100 rows from the table then value of 100 should be updated into the incremental (metadata) table. So, once the value is updated into the database then next day new rows will be copied to the output (i.e. copy from 101).

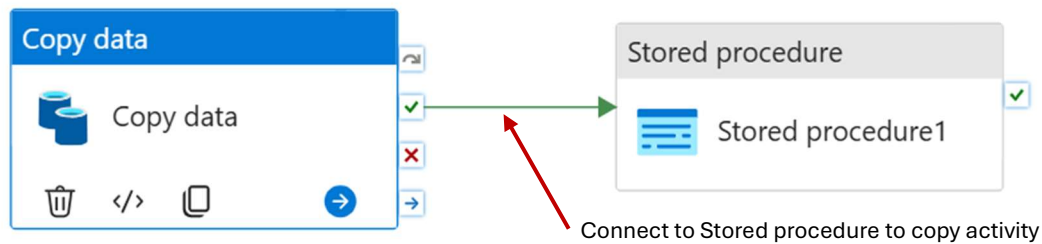
	Inc_ID	IncTable	IncSchema	IncFolderPath	IncColumn	IncValue	
1	1	Customers	dbo	dbo/Customers	CustomerID	0	IncValue should be updated into db with the maximum output value

- ➔ To achieve this, we will create a stored procedure in the database by passing two arguments 'IncValue' and 'IncTable'

```
create proc set_inc_value
@IncValue Int,
@IncTable Varchar (100)
as
update IncrementalTable
set IncValue = @IncValue
where IncTable = @IncTable
```

This stored procedure will set the incremental value and updates the table as per the inputs.

- ➔ Now, in pipeline inside the ForEach activity add one more activity named **Stored Procedure** to update the Incremental value into the database from the output.
- ➔ Connect the copy data to the stored procedure as per below image.



Configuration of stored procedure:

- ➔ Connect the same database configuration used in before activity
- ➔ Under the Stored procedure name drop down select the stored procedure method created in the database.
- ➔ As we created stored procedure by passing 2 arguments, we must pass arguments for the activity.

Note: arguments name should be same as created in the database

Before entering the arguments, value and input create another activity for getting the maximum ID value of the table.

As we cannot get the Incremental value of the IncrementalTable (meta data table) from the copy activity. So, we will create a lookup activity on success to copy activity to look for the IncValue in the meta data table.

Step 5: Create a lookup Activity for IncValue.

- ➔ Name the activity name as 'LookUp Max Value'
- ➔ Under the settings load the same data set used for first LookUp activity, but here we will use query-based search for the maximum ID present in the table
- ➔ Query from the database is "`select max(CustomerID) as max_value from dbo.Customer`"
 - CustomerID → IncColumn
 - Customer → IncTable
 - dbo → IncSchema

General Settings User properties

Source dataset * INC_SQL_DB_Setup Open + f

Dataset properties ⓘ

Name	Value
TableName	""
SchemaName	""

First row only ☒

Use query ☐ Table ☒ Query ☐ Stored procedure

Query *

Enter value dynamically here

Enabled because only one value is required

Expression for getting the maximum value based on ID value

Pipeline expression builder

Add dynamic content below using any combination of expressions, functions and s

```
select max(@{item().IncColumn}) as max_value from @{item().IncSchema}.@{item().IncTable}
```

As per the above image values are entered dynamically and then expression used to get the maximum value.

- Item().IncColumn refers to Column name
 - Item().IncSchema refers to Schema name
 - Item().IncTable refers to Table name
 - @ expression refers to parameter
- ➔ Once maximum value from the lookup activity output is out then the value should be passed into the stored procedure to store this value into the table.

Step 6: Time to add the stored procedure arguments

Below is the screen shot of the settings for the stored procedure,

- ➔ Select name of the procedure from the dropdown menu.
- ➔ Enter the parameters names as same as the arguments created in the database.
- ➔ Now, enter the table name dynamically as below by selecting from the lookup output.

General **Settings** User properties

i To reference SQL pool, use the SQL pool stored procedure instead.

Linked service * **i** AzureSqlDatabase1 Test connection

Integration runtime * AutoResolveIntegrationRuntime Edit

Stored procedure name * [dbo].[set_inc_value]
☒ Enter manually

▼ Stored procedure parameters **i**

← Import + New | Delete

Select appropriate data type when created in db

☐	Name	Type *	Value
☐	IncValue	Int64	@activity('Lookup Max Value').outu...
☐	IncTable	String	@item().IncTable

- ➔ When entering the dynamic value under the pipeline expression builder select activity outputs tab.
- ➔ Under the activity outputs tab look for the previous lookup activity outputs (i.e. LookUp Max Value)

- ➔ Select the Lookup Max Value first row which represents the data of the first row (i.e. max_value)
- ➔ Now, followed by the selected activity output enter “.max_value” which represents the output of the max_value

Pipeline expression builder

Add dynamic content below using any combination of [expressions](#), [functions](#) & [variables](#)

`@activity('Lookup Max Value').output.firstRow.max_value`

Clear contents

	Activity outputs	Parameters	System variables
ForEach iterator			
INC_Lookup count			
Count of the row			
INC_Lookup value array			
Array of row data			
Lookup Max Value			
Lookup Max Value activity output			
Lookup Max Value first row			
Data of the first row			

Type max_value to get the maximum ID value created in previous step

max_value represents the maximum value from the expression → `select max(CustomerID) as max_value from dbo.Customer`

- ➔ Once the pipeline is created then validate and run the pipeline.

After running the pipeline, observe the output data for clear understanding on the data how it is transmitted and stored.

To test the incremental pipeline now add the new data into the table and run the pipeline, you will observe the new file will be created with only the new data.